

Whatsapp

Ernesto Rousell Zurita
Claudia Puentes Hernández

Enero 2025

1 Arquitectura

Para implementar el sistema de mensajería, nuestra idea se basa en una arquitectura descentralizada utilizando Chord. Este modelo organiza los datos en la red para que estén distribuidos entre servidores y clientes, asegurando escalabilidad, resistencia a fallos y un funcionamiento eficiente.

En cuanto a los clientes, la propuesta es que actúen como entidades responsables de enviar y recibir mensajes. Los mensajes entregados se guardarían en bases de datos SQLite locales, lo que permitiría a los usuarios acceder a su historial sin depender continuamente de los servidores. Para los mensajes no entregados, los servidores almacenarían estos de manera temporal hasta que el destinatario pueda recibirlos.

La distribución de servicios se realizaría utilizando dos redes separadas en Docker:

Red de clientes: Alojaría las instancias de los clientes y estaría configurada para comunicarse con los servidores mediante solicitudes HTTP. Esta red es independiente para facilitar la escalabilidad y evitar interferencias.

Red de servidores: Aquí se organizarían los servidores en el anillo Chord. Un servidor principal inicializaría la red y los servidores secundarios se unirían, formando una estructura descentralizada. Esta red sería responsable de la replicación de datos, sincronización de mensajes pendientes y redistribución de responsabilidades en caso de fallos.

La comunicación entre estas redes se gestionaría mediante reglas de enrutamiento configuradas en Docker, asegurando que los clientes puedan acceder a los servicios ofrecidos sin comprometer la seguridad ni la organización.

Los servidores estarían organizados en una red distribuida basada en Chord, teniendo varias funciones clave. Inicialmente, se crearía un servidor principal que establece la red Chord, a la cual se conectan los servidores secundarios. Estos servidores secundarios localizan al servidor principal y se unen a la red,

formando una estructura descentralizada. Cada servidor almacena información primaria y secundaria para asegurar la redundancia y la recuperación de datos en caso de fallos.

En cuanto al gestor de identidad, no se implementaría como una entidad separada, sino que su funcionalidad está integrada en los servidores. Estos manejarían la autenticación y el registro de usuarios utilizando una base de datos distribuida. Esta base de datos tendría información como nombres de usuario, contraseñas y claves necesarias para la autenticación. Todos los servidores tendrían acceso a esta base de datos, y su replicación asegura que cualquier servidor pueda manejar solicitudes de autenticación de manera eficiente.

Con esta arquitectura, se espera minimizar los puntos únicos de fallo y cumplir con un nivel de tolerancia a fallos igual a 2, replicando datos críticos entre al menos tres nodos. Esto garantizaría que, incluso si dos servidores fallan, el sistema pueda seguir funcionando hasta que los servicios se restablezcan. Además, el diseño busca que los usuarios puedan enviar mensajes sin interrupciones, mejorando la experiencia general.

2 Procesos

Para implementar los procesos del sistema, nuestra idea se basa en asignar responsabilidades claras entre los clientes y los servidores.

Los clientes manejarán principalmente la interfaz gráfica, donde los usuarios podrán enviar y recibir mensajes de manera sencilla. Una vez entregados, los mensajes se almacenarán localmente en bases de datos SQLite.

En cuanto a los servidores, queremos que realicen las tareas más complejas. La propuesta es que el servidor principal configure la red Chord inicial, mientras que los servidores secundarios se conecten a esta red automáticamente al iniciarse. Una vez integrados, los servidores manejarán tareas como:

- Gestión de identidades mediante una base de datos distribuida.
- Almacenamiento de mensajes pendientes hasta que el destinatario pueda recibirlos.
- Replicación de datos para garantizar disponibilidad y recuperación en caso de fallos.

Para procesar solicitudes, planeamos implementar un modelo basado en hilos. Esto permitirá que cada servidor maneje múltiples solicitudes al mismo tiempo, ya sea de clientes o de otros servidores. Este enfoque asegura que el sistema mantenga un rendimiento óptimo incluso en momentos de alta demanda.

En general, los procesos estarán diseñados para ser ligeros, eficientes y resistentes

a fallos, asegurando que tanto los clientes como los servidores puedan cumplir sus roles incluso en condiciones adversas.

3 Comunicación

Para implementar la comunicación en el sistema de mensajería distribuida, nuestra idea es utilizar HTTP como protocolo principal. Los clientes enviarían mensajes a los servidores utilizando solicitudes POST, mientras que los mensajes pendientes se recuperarían mediante GET. Una vez que los mensajes se entreguen, se guardarían localmente en la base de datos SQLite del cliente.

En cuanto a la comunicación entre servidores, queremos que también utilice HTTP. Esta comunicación será clave para que los servidores coordinen tareas como la sincronización de datos, la replicación de mensajes pendientes y la redistribución de responsabilidades en caso de fallos. Además, los servidores compartirían información sobre el estado de los nodos y decidirían cómo promover datos replicados a principales.

Dentro de cada servidor, los procesos necesitarán comunicarse entre sí. La propuesta es que esto se gestione mediante estructuras de datos compartidas y mecanismos de sincronización como semáforos o bloqueos. Esto aseguraría que las operaciones concurrentes no entren en conflicto. Por ejemplo, cuando un servidor recibe una solicitud para almacenar un mensaje pendiente, coordinaría su escritura y replicación sin interferir con otras solicitudes que puedan estar procesándose.

Si bien el modelo actual está diseñado para funcionar con HTTP, pensamos que en el futuro sería interesante explorar un modelo basado en sockets, especialmente para la comunicación en tiempo real. Esto podría ser beneficioso en sistemas con alta concurrencia, donde la latencia debe mantenerse al mínimo.

4 Coordinación

Para implementar la coordinación en el sistema, nuestra idea es garantizar que los datos sean accesibles y consistentes en toda la red, incluso en caso de fallos. Los servidores sincronizarán los mensajes pendientes y las claves de identidad entre los nodos del anillo Chord. Cuando un cliente envíe un mensaje, el servidor lo almacenará temporalmente y replicará en al menos dos nodos adicionales. De esta forma, si un servidor falla, las copias replicadas permitirán que otro nodo tome la responsabilidad de los datos sin interrupciones.

El acceso a los datos se organizará siguiendo un proceso definido. Cada nodo verificará primero si la clave solicitada está dentro de su rango de responsabilidad. Si no lo está, reenviará la solicitud al nodo más cercano utilizando su tabla

de enrutamiento. Esto permitirá localizar los datos de manera eficiente con un número reducido de saltos.

La toma de decisiones será completamente descentralizada. Por ejemplo, cuando se detecte un fallo, los nodos que almacenan copias replicadas promoverán automáticamente esos datos a copias principales y notificarán a otros nodos relevantes para que actualicen sus tablas de enrutamiento. Este proceso asegurará que las operaciones puedan continuar sin depender de un nodo central.

Para evitar condiciones de carrera al manejar datos replicados, los servidores emplearán mecanismos como bloqueos distribuidos. Estos bloqueos garantizarán que, si varios nodos intentan modificar los mismos datos al mismo tiempo, solo uno pueda hacerlo. Además, se sincronizarán las acciones mediante mensajes de confirmación entre nodos, asegurando que las actualizaciones se completen antes de permitir nuevas operaciones sobre los mismos datos.

Estos mecanismos permitirán que el sistema mantenga su integridad y disponibilidad incluso en condiciones adversas. Aunque los mensajes entregados no dependen de los servidores porque están almacenados localmente en SQLite en los clientes, los servidores deberán coordinarse eficazmente para gestionar los mensajes pendientes y mantener actualizadas las claves de identidad.

5 Nombrado y Localización

Para implementar el nombrado y la localización en nuestro sistema, planeamos utilizar claves hash para identificar los datos y ubicarlos en la red Chord. Cada servidor se encargará de gestionar un rango específico de claves, lo que hará que sea sencillo identificar dónde se encuentran los mensajes pendientes y las claves de identidad. Una vez que los mensajes sean entregados, estos ya no dependerán de los servidores porque estarán almacenados de forma local en las bases de datos SQLite de los clientes.

Cuando un cliente necesite enviar un mensaje pendiente, nuestra idea es que consulte a un servidor, que, mediante su tabla de enrutamiento, reenviará la solicitud al servidor que tenga la información necesaria. Este proceso aprovechará la estructura eficiente de Chord, permitiendo localizar los datos con pocos saltos y garantizar búsquedas rápidas.

6 Consistencia y Replicación

Aquí nuestra idea es centrarnos en proteger los datos críticos, como los mensajes pendientes y las claves de identidad. Cada dato tendría una copia principal (o primaria) y al menos dos réplicas (copias secundarias) distribuidas en otros nodos del anillo Chord. Esto garantizaría que los datos críticos estén protegidos

contra fallos, ya que si un nodo falla, las réplicas permitan que otro servidor asuma la responsabilidad de los datos de forma inmediata.

El nivel de tolerancia a fallos que estamos considerando es 2, por lo que se necesitarían un mínimo de tres nodos para almacenar los datos replicados. Así, incluso si dos servidores fallan al mismo tiempo, el sistema podría continuar operando, ya que habría un nodo con toda la información necesaria hasta que los servidores caídos se recuperen.

La replicación se organizaría de manera que cada servidor maneje datos únicos como principal, pero también almacene réplicas de los datos de otros nodos. Por ejemplo, si hay varios servidores en la red, cada uno almacena su porción específica y copia los datos de otros k servidores, según el esquema de replicación definido. Cuando un servidor detecta un fallo, promueve las réplicas que tiene a datos principales y actualiza las tablas de enrutamiento para notificar a los demás servidores sobre el cambio.

Este enfoque aseguraría que el sistema pueda mantener la consistencia de los datos y evitar interrupciones, todo mientras minimiza la sobrecarga innecesaria al replicar únicamente los datos más relevantes.

7 Tolerancia a Fallos

Para manejar la tolerancia a fallos en el sistema es asegurar que, incluso si una o varias partes fallan, el sistema pueda seguir funcionando con la menor interrupción posible. Esto lo logramos mediante la replicación de datos críticos y una arquitectura descentralizada basada en Chord. Queremos que los datos y las responsabilidades estén distribuidos entre varios nodos para que un único fallo no afecte al sistema completo.

Si un servidor falla, los nodos que tienen las réplicas de sus datos asumirán automáticamente la responsabilidad de estos, promoviendo las copias replicadas a datos principales. Los mensajes que ya han sido entregados no se ven afectados porque están almacenados localmente en los clientes, quienes son responsables de su propio historial de mensajes.

El sistema está diseñado para tolerar hasta dos fallos simultáneos. Esto significa que los datos críticos estarán replicados en al menos tres nodos. Incluso si dos servidores dejan de funcionar, el tercero podrá mantener los datos necesarios para que el sistema siga operativo hasta que los servidores caídos puedan reintegrarse.

También queremos que el sistema sea capaz de manejar fallos parciales, como servidores que se desconectan temporalmente. Las réplicas asegurarán que la red pueda continuar operando sin perder información. Cuando un nodo que

estuvo caído se reincorpore al sistema, sincronizará su estado con los nodos que tienen los datos principales. Por otro lado, los nodos nuevos que se integren al sistema serán asignados al anillo Chord, recibiendo un rango de claves y copias de datos para equilibrar la carga

Con esta estructura, buscamos no solo asegurar la continuidad del servicio, sino también minimizar las pérdidas de datos y optimizar el uso de los recursos cuando se agregan o eliminan nodos de manera dinámica.

8 Seguridad

La seguridad en el sistema se pudiera basar en múltiples capas de protección, incluyendo el cifrado de las comunicaciones, la autenticación de usuarios y la protección de los datos almacenados tanto en los clientes como en los servidores. Las bases de datos SQLite en los clientes podrían cifrarse para proteger los mensajes almacenados localmente, asegurando que solo el usuario autorizado pueda acceder a ellos. Esto sería particularmente importante en escenarios donde los dispositivos se pierden o son comprometidos.

La comunicación entre clientes y servidores utiliza cifrado de clave pública y privada, lo que evita que terceros puedan interceptar o modificar los mensajes en tránsito. Este cifrado también protege las credenciales de autenticación durante las conexiones iniciales, reduciendo el riesgo de robo de información sensible.

Para fortalecer la seguridad, se podrían implementar auditorías periódicas que permitan evaluar posibles vulnerabilidades en el sistema. Además, un sistema de registro centralizado podría monitorizar intentos de acceso no autorizado, patrones inusuales de actividad o errores en la comunicación. Este tipo de monitoreo ayudaría a identificar y mitigar ataques antes de que causen problemas graves.

Otro punto importante sería considerar la implementación de un sistema de respaldo para los mensajes locales almacenados en SQLite. Aunque actualmente no se respalda esta información, ofrecer una opción de copia de seguridad cifrada podría mejorar la experiencia del usuario y garantizar que los mensajes no se pierdan definitivamente en caso de fallos o pérdida del dispositivo. Por último, mecanismos de renovación periódica de claves y cifrado de extremo a extremo podrían añadirse para reforzar aún más la seguridad del sistema en el futuro.